ShiftSense — Installation, Deployment and Operations

Version 1.0 · October 2026. Commands run from `application/`.

Contents

- 1. Local installation (SQLite)
- 2. Pilot deployment (Docker)
- 3. Configuration
- 4. Moving from SQLite to PostgreSQL
- 5. Scheduled jobs (without Docker)
- Backup and restore
- Monitoring and health checks
- Troubleshooting
- Incident response

1. Local installation (SQLite)

Requirements: Python 3.11+ (tested on 3.13), about 1 GB disk.

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
scripts/run_app.sh            # = .venv/bin/python -m streamlit run app.py
```

Open http://localhost:8501. The first start builds the synthetic dataset and the published roster (about 10 s) and saves it to `data/shiftsense.db`. Streamlit runs with `fileWatcherType = "none"`: **restart the app after any code change.**

Tests: `scripts/run_tests.sh` (about 770 tests, 15 minutes on a laptop).

2. Pilot deployment (Docker)

```
cp .env.example .env                                  # fill in; never commit it
mkdir -p .streamlit && cp .streamlit/secrets.toml.example .streamlit/secrets.toml   # SSO settings
docker compose --env-file .env up -d --build
```

Service	Role
`db`	PostgreSQL 16, no public port, volume `pgdata`
`app`	ShiftSense (Streamlit)
`jobs`	nightly verified backup (keep 14) and restore drill; health check every 15 min with alert email
`proxy`	Caddy, automatic HTTPS for `SHIFTSENSE_DOMAIN`

Before go-live: set `SHIFTSENSE_ENV=pilot`, configure SSO (`SHIFTSENSE_AUTH=sso` or `both`), set the SMTP relay and `SHIFTSENSE_ALERT_EMAIL`, remove or change the demo accounts, and run `scripts/health.sh` until every essential check passes. Copy backups off the host.

3. Configuration

Variable	Purpose	Default
`SHIFTSENSE_ENV`	`demo` (demo accounts listed at sign-in) or `pilot`	`demo`
`SHIFTSENSE_DB`	SQLite file	`data/shiftsense.db`
`SHIFTSENSE_DATABASE_URL`	PostgreSQL URL (PostgreSQL used when set)	not set
`SHIFTSENSE_PG_SCHEMA`	PostgreSQL schema	`public`
`SHIFTSENSE_AUTH`	`password`, `sso` or `both`	`password`
`SHIFTSENSE_BREAKGLASS`	emergency password account(s) in SSO mode	not set
`SHIFTSENSE_IDLE_MINUTES` / `_LOCKOUT_ATTEMPTS` / `_LOCKOUT_MINUTES`	session and lockout	30 / 5 / 15
`SHIFTSENSE_PBKDF2_ITERATIONS`	password hashing cost	600000
`SHIFTSENSE_EMAIL`, `SMTP_*`, `SHIFTSENSE_EMAIL_FROM`	optional email copies	off
`SHIFTSENSE_ALERT_EMAIL`	health alert recipient	not set
`SHIFTSENSE_PHOTO_MAX_MB`	profile photo limit	5
`ANTHROPIC_API_KEY`, `SHIFTSENSE_AI`, `SHIFTSENSE_AI_CONVERT`	optional AI	off without a key
`SHIFTSENSE_LOG`	log file	`data/shiftsense.log`

Business configuration (read-only in the app): `config/policy.yaml` (rules, queues, shifts, costs, thresholds) and `config/users.yaml` (roles, permissions, demo accounts).

4. Moving from SQLite to PostgreSQL

`scripts/migrate_to_postgres.sh` takes a verified SQLite backup first, refuses a non-empty target unless `--replace` is given, copies every table with its ids in one transaction, and verifies the audit chain and row counts.

5. Scheduled jobs (without Docker)

`scripts/ops-examples/cron.example` and `scripts/ops-examples/systemd/` contain the nightly backup and drill, the drop-folder staging and the 15-minute health check. Secrets go in `/etc/shiftsense/env`, never on command lines.

Backup and restore

The same command works for SQLite and PostgreSQL (whichever is configured). Run from `application/`.

Task	Command
Create a verified backup, keep the newest 14	`scripts/backup.sh create --keep 14`
Verify a backup	`scripts/backup.sh verify <file>`
Restore	`scripts/backup.sh restore <file>`

Task	Command
Restore drill (into a throw-away copy/schema, checked, then dropped)	`scripts/backup.sh drill [<file>]`

SQLite. `create` uses SQLite's online backup API (safe while the app runs) and writes `data/backups/shiftsense-YYYYmmdd-HHMMSS.db`, then verifies it: integrity check, workspace loads, audit chain intact.

PostgreSQL. `create` writes `shiftsense-pg-*.zip`: one CSV per table and a manifest from one consistent snapshot (no `pg_dump` needed). `restore` takes a safety backup first, then reloads everything in one transaction.

Restore procedure 1. Announce downtime. Stop the app (`docker compose stop app jobs`, or stop the Streamlit process). 2. `scripts/backup.sh verify <file>`; do not continue if it reports problems. 3. `scripts/backup.sh restore <file>` (a safety copy of the current data is taken automatically). 4. Start the app; run `scripts/health.sh` and confirm *audit chain verified* and *0 rule violations*. 5. Record who restored, which file and why **outside the app**: a restore also replaces the audit log with the backup's version.

Schedule (Docker stack). The `jobs` container runs the nightly backup with `--keep 14` and the restore drill. In pilot mode the health check fails when the newest backup is older than 26 h. Copy backups off the host; they contain accounts and contact details and must be stored as confidential.

Moving SQLite → PostgreSQL. `scripts/migrate_to_postgres.sh` (takes a verified SQLite backup first, refuses a non-empty target without `--replace`, copies every table in one transaction, verifies the audit chain and row counts).

Monitoring and health checks

```
scripts/health.sh            # human-readable; exit code 1 if an essential check fails
scripts/health.sh --json     # for monitoring tools
scripts/health.sh --alert    # emails SHIFTSENSE_ALERT_EMAIL on failure, at most once a day per
failure
```

Check	Fails when
Database	file/connection missing; SQLite `integrity_check` not ok
Policy	`config/policy.yaml` does not load
Roles	not exactly the six roles (admin, agent, team_lead, ops_head, analyst, hr)
Accounts	no active admin; unknown roles
Live workspace	does not load, or has any hard-rule violation
Audit trail	hash chain broken
Optimiser	OR-Tools cannot be imported
Backups (pilot mode)	newest backup older than 26 h
Sign-in (pilot mode)	password-only sign-in (`SHIFTSENSE_AUTH=password`); weak password hashing; a demo password still in use
Demo scenario (pilot mode)	a demo scenario is still loaded
Email / AI	reported as configured or not (never the secret itself)
Disk	less than 0.5 GB free

Docker stack. The `jobs` container runs the health check every 15 minutes with `--alert`.

Logs. `data/shiftsense.log` (rotating, `SHIFTSENSE_LOG`): failed actions with traceback (rolled back) and permission refusals (`permission denied user=… role=… needs=…`). It never contains passwords, tokens or message text. Docker: `docker compose logs app`.

What to watch in the app. Admin → System status; Ops → Overview (open alerts, escalations, carried risk).

Troubleshooting

Symptom	Likely cause	What to do
`ModuleNotFoundError: ortools` (or other module)	dependencies not installed	`.venv/bin/pip install -r requirements.txt`
"Can't pickle … World is not the same object"	code changed while the app was running	restart the app (the failed action changed nothing)
Code change has no effect	`fileWatcherType = "none"`	restart the app
"Port 8501 is not available"	another instance is running	stop it, or use `--server.port`
Health: audit chain broken	an audit row was edited outside the app	restore the last verified backup; investigate file/database access
Planner: "No roster can meet every hard rule…"	infeasible inputs (frozen days, leave, limits)	read the listed blocking rules; change inputs, never the hard rules
Planner: "Conflict: the published roster changed…"	live roster changed after the draft	create a new draft
"Someone else changed this workspace…"	another save happened first	the latest version is loaded; repeat the action
Decision: "already handled by …"	someone acted first	close the card; the screen shows the latest state
Screen looks out of date	screens do not auto-refresh	sidebar → **Refresh**
Account locked	5 wrong passwords	wait 15 min, or Admin → account card → Unlock
SSO: "no active account for this email"	account missing/inactive or email differs	Admin creates/activates the account with the same email
Email "not configured"	`SHIFTSENSE_EMAIL`, `SMTP_HOST` or `SHIFTSENSE_EMAIL_FROM` missing	set them, or leave email off (in-app still works)
Email "not sent"	in-app preference, unverified address, or demo scenario	reason is shown next to the status
Import rows held back	mandatory field missing or invalid value	download *rows to fix*, correct, re-upload
Photo shows initials	photo missing, invalid or not visible to you	HR → profile → Photo; check permissions
Theme switch does nothing	Streamlit upgraded past 1.64	pin Streamlit 1.64 or update `ui_kit.apply_theme`

Incident response

Severity: **P1** app down or data integrity at risk · **P2** a workflow blocked for a role · **P3** cosmetic / single user.

Step	Action
1. Detect	Health alert email, `scripts/health.sh`, user report, `data/shiftsense.log`

Step	Action
2. Contain	P1: stop the app (`docker compose stop app`) to prevent further writes. Security: deactivate affected accounts (Admin) and rotate secrets in `.env` / `secrets.toml`
3. Preserve	Take a backup **before** changing anything (`scripts/backup.sh create`); copy the log
4. Diagnose	Health output; log tracebacks; audit trail (Admin → Audit) for who did what
5. Recover	Fix configuration or restore the last verified backup (see `backup-and-restore.md`); never edit audit rows
6. Verify	`scripts/health.sh` all essential checks pass; spot-check the published roster and open alerts
7. Communicate	Tell affected roles; staffing decisions made outside the app during downtime must be entered afterwards
8. Review	Record cause, timeline and fix; add a test for the failure

Specific cases - **Audit chain broken:** treat as a security incident. Do not "repair" rows; restore and investigate access. - **Suspected account compromise:** deactivate, reset password (Admin, reason), review the account's audit rows. - **Wrong roster published:** Planning → restore an earlier version → validate → publish (rollback is a normal, audited plan action). - **Wrong data import:** apply a corrected import (new version); reference data is versioned, history is kept. - **Personal data exposure:** stop sharing, record what and to whom, notify the data owner (EXL HR/legal).